

QUANT FINANCE TERMINAL ·  
v2.046

SYS://CURRICULUM/D051.MD · PHASE 2 ●  
LIVE

LECTURE · 量化金融 365

# DAY 051 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

## 装饰器与上下文管理器

// Decorators & ContextManagers

KEY CONCEPTS · 三大要点

- ▶ 计时装饰器
- ▶ 重试装饰器
- ▶ timer 上下文

01 · WHY THIS LESSON · 这节课为什么重要

## 本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面我们学了不少分析工具,这一节专门解决三件天天要干的烦心事。想象给一个函数套个壳,壳上加了新功能,函数本身一个字没改,这就是装饰器。量化里有三件横切的事最常见:一是回测函数跑得慢,想知道到底耗了多久;二是联网拉行情偶尔断线,想让它自动重试几次;三是同样的数据被反复计算,想把上次的结果缓存起来直接拿。这三件事如果在每个函数里复制粘贴,改一处要改十处,又乱又容易出错。这节我们用装饰器和 with 块,把计时、重试、缓存一次写好,到处复用,代码立刻干净10倍。还会拿招商银行的真实日线数据,让你亲眼看到缓存把耗时砍到接近零。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解装饰器就是给函数套个壳加新功能,原函数一行原码都不用改
- 02 会写 @timer 自动计时、会写 @retry 让联网失败自动重试几次
- 03 会用 functools.lru\_cache 把重复计算的结果缓存起来,第二次直接秒回
- 04 会用 contextlib.contextmanager 配 with 块,进门自动准备、出门自动收尾
- 05 知道 functools.wraps 为什么重要:不加它,被装饰后函数会丢掉自己的名字

02 · CONTEXT · 历史与背景

# 老岳的回测脚本里到处复制粘贴计时和重试， 改一处要改十处

// 不读历史的策略走不远

老岳会一点 python,自己写了套回测脚本。一开始他想知道每个步骤跑多久,就在每个函数开头记一个时间、结尾再记一个时间、相减打印出来。后来又发现联网拉数据偶尔会断,他又在每个拉数据的地方手写一段 try except 加循环重试。脚本越写越长,光是这些计时和重试的代码就重复了十几处。有一天他想把打印格式改一下、把重试次数从 2 次调成 3 次,结果要在十几个地方一个一个改,漏改了两处还跑出 bug,折腾了一下午。后来他学了装饰器,恍然大悟:原来这些跟业务无关、却到处都要的事,可以抽出来写一次,然后在需要的函数上面贴一行 @timer 或者 @retry 就行了。原函数一个字没动,功能却自动有了。改格式只改装饰器那一处,所有贴了它的函数全都跟着变。老岳的脚本一下子瘦了一大圈,清爽得他自己都不敢信。

# 把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

## CONCEPT\_01

### 装饰器是什么:在函数上面写一行就自动加功能

装饰器就是给函数套个壳。你只要在某个函数上面写一行 `@timer`,这个函数被调用时就自动会计时了,而函数里面的代码一个字都没改。举个最小的例子:你有个函数叫 `backtest`,平时直接跑;现在在它上面加一行 `@timer`,再跑它,屏幕上就会自动多打印一句『`backtest` 耗时 800 毫秒』。计时这件事被装饰器悄悄加上了,原来的 `backtest` 还是那个 `backtest`。这就像给手机套个壳,壳加了防摔功能,手机本身没动。

## CONCEPT\_02

### `functools.wraps`:不加它,被装饰后函数会丢掉名字

写装饰器有个一定要记住的小细节:在壳的里层函数上面加一行 `@functools.wraps(func)`。为什么?因为套壳之后,外界看到的其实是壳,如果不加这一行,你打印 `backtest.__name__` 会发现它的名字变成了壳的名字 `wrapper`,原来的 `backtest` 这个名字丢了,以后 `debug`、看日志都对不上号。加了 `functools.wraps`,被装饰后的函数依然保留原来的名字和说明,`debug` 时不抓瞎。这一行几乎是写装饰器的标配,养成习惯就好。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT\_03

## @retry:联网失败自动再试几次,救场神器

拉行情数据最烦的就是网络偶尔抖一下,一次没拉到整个脚本就崩了。@retry 装饰器解决这个:给函数套上它,函数一旦失败就自动再试,试够指定次数还不行才真正报错。举个例子:设

@retry(times=3),第 1 次断网失败、它自动等一下再试第 2 次又失败、第 3 次成功了,你的脚本就当无事发生继续跑。本节代码里我们故意做了个前 2 次失败、第 3 次成功的函数,让你亲眼看到 retry 怎么把脚本从崩溃边缘救回来。

CONCEPT\_04

## functools.lru\_cache:同样输入第二次直接秒回

有些计算很慢,比如联网拉一只票两年的日线再算均线,要花不少时间。如果同样的参数你算了一遍、待会儿又要算,那就白白等一遍。lru\_cache 就是给函数加一个『记忆』:它会把每次调用的输入和结果记下来,下次遇到一模一样的输入,直接把上次的结果端出来,根本不重新算。举个例子:第一次调用花了 2 秒联网计算,第二次同样参数调用几乎是 0 秒。本节用招商银行真实数据演示,你能看到第二次耗时被砍到接近零。

#### CONCEPT\_05

### contextmanager 配 with:进门开灯,出门关灯

上下文管理器就是那个 with 块。它的精髓是进门自动准备、出门自动收拾,就像走进房间自动开灯、离开房间自动关灯,你不用记得手动关。最常见的就是 with open(文件) 读完自动关文件。用 contextlib.contextmanager 你可以自己写一个:函数里写一个 yield,yield 之前的代码是『进门开灯』(比如开始计时),yield 之后的代码是『出门关灯』(比如打印耗时)。一个 yield 把代码劈成开灯和关灯两半,with 块结束时关灯那半自动执行,资源用完自动收尾,绝不会忘。

04 · PYTHON LAB · 真实可跑代码

# 装饰器与上下文管理器 — functools.wraps 写 @timer / @retry 重试 / lru\_cache 缓存 / contextmanager 写 with 计时块

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_051_decorator_context.py - 装饰器与上下文管理器:给函数套壳加功能,不改原码
# 真名上屏:装饰器 functools.wraps / @timer 计时 · @retry 重试 · functools.lru_cache 缓存
# 真名上屏:contextlib.contextmanager + with 语句 + yield(进门开灯 / 出门关灯)
import time
import functools
import random
import contextlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

pd.set_option('display.width', 160)
plt.rcParams['axes.unicode_minus'] = False
random.seed(42)
np.random.seed(42)

# ==== 1. @timer 计时装饰器:给函数套壳,自动测耗时,原函数一个字没改 ====
# functools.wraps 保留被装饰函数的原名字,debug 时不抓瞎
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/6)

# 装饰器与上下文管理器 一

## functools.wraps 写 @timer / @retry 重试 / lru\_cache 缓存 / contextmanager 写 with 计时块

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
cost = time.perf_counter() - t0
print(f'[计时] {func.__name__} 耗时 {cost*1000:.1f} 毫秒')
return result
return wrapper

# ==== 2. @retry 重试装饰器:失败自动再试,带简单退避,全失败才抛出 ====
def retry(times=3, delay=0.2):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_err = None
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last_err = e
                    print(f'[重试] 第 {attempt} 次失败:{e},{delay} 秒后再试')
                    time.sleep(delay)
            raise RuntimeError(f'重试 {times} 次仍失败') from last_err
        return wrapper
    return decorator

# 演示 retry:前 2 次故意失败、第 3 次成功(确定性计数器,不用随机,方便复现)
_attempt_log = []
_counter = {'n': 0}
@retry(times=3, delay=0.05)
def flaky_fetch():
    _counter['n'] += 1
    ok = _counter['n'] >= 3
    _attempt_log.append((_counter['n'], ok))
```



04 · PYTHON LAB · 真实可跑代码 · 续 (3/6)

# 装饰器与上下文管理器 一

## functools.wraps 写 @timer / @retry 重试 / lru\_cache 缓存 / contextmanager 写 with 计时块

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
if not ok:
    raise ConnectionError('网络抖动')
return '拉到数据'

print('==== 1+2. retry 重试演示(前 2 次失败,第 3 次成功)====')
print('最终结果:', flaky_fetch())
print('每次尝试记录:', _attempt_log)

# ==== 3. functools.lru_cache 缓存:同样的输入第二次直接拿上次结果 ====
# 慢函数:真的用 baostock 拉招商银行 sh.600036 日线 + 算移动平均
@functools.lru_cache(maxsize=8)
def load_and_ma(code, start, end, window):
    lg = bs.login()
    if lg.error_code != '0':
        raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
    rs = bs.query_history_k_data_plus(code, 'date,close', start_date=start,
    end_date=end, frequency='d')
    rows = []
    while rs.error_code == '0' and rs.next():
        rows.append(rs.get_row_data())
    bs.logout()
    df = pd.DataFrame(rows, columns=['date', 'close'])
    df['close'] = pd.to_numeric(df['close'])
    ma = df['close'].rolling(window).mean()
    return float(ma.dropna().iloc[-1]), len(df)

print('\n==== 3. lru_cache 缓存:第一次联网慢,第二次命中缓存几乎瞬间 ====')
CODE, NAME = 'sh.600036', '招商银行'
START, END = '2023-01-01', '2024-12-31'
t0 = time.perf_counter()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/6)

# 装饰器与上下文管理器 一

## functools.wraps 写 @timer / @retry 重试 / lru\_cache 缓存 / contextmanager 写 with 计时块

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ma1, n1 = load_and_ma(CODE, START, END, 20)
cost_first = time.perf_counter() - t0
t0 = time.perf_counter()
ma2, n2 = load_and_ma(CODE, START, END, 20) # 同样参数,直接命中缓存
cost_cached = time.perf_counter() - t0
print(f'{NAME} 共 {n1} 个交易日,20 日均线最新值 {ma1:.2f} 元')
print(f'第一次(无缓存,需联网)耗时 {cost_first*1000:.1f} 毫秒')
print(f'第二次(命中 lru_cache)耗时 {cost_cached*1000:.3f} 毫秒')
speedup = cost_first / cost_cached if cost_cached > 0 else float('inf')
print(f'缓存把耗时砍到约 1/{speedup:.0f},几乎瞬间返回')

# ==== 4. contextlib.contextmanager:用 with 块自动收尾(进门开灯,出门关灯)====
# yield 前是进门(开灯),yield 后是出门(关灯);with 块结束自动执行 yield 之后的代码
@contextlib.contextmanager
def timer_block(name):
    print(f'[with] 进门:开始计时「{name}」')
    t0 = time.perf_counter()
    try:
        yield
    finally:
        cost = time.perf_counter() - t0
    print(f'[with] 出门:「{name}」耗时 {cost*1000:.1f} 毫秒')

# ==== 5. 确定性 CPU 慢函数:对固定大数组反复算移动平均求和(不用随机,可复现)====
def heavy_compute(size, rounds=30):
    arr = np.arange(size, dtype=float)
    total = 0.0
    for _ in range(rounds):
        ma = np.convolve(arr, np.ones(50) / 50, mode='valid')
        total += float(ma.sum())
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/6)

# 装饰器与上下文管理器 一

## functools.wraps 写 @timer / @retry 重试 / lru\_cache 缓存 / contextmanager 写 with 计时块

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
return total

print('\n==== 4+5. with 计时块 包住一段 CPU 计算 ====')
with timer_block('算指标'):
    val = heavy_compute(100000)
print(f'计算结果(可复现):{val:.2e}')

# 给 heavy_compute 也套上 @timer, 测不同数据量下的耗时增长
timed_heavy = timer(heavy_compute)
print('\n==== 6. @timer 测不同数据量下慢函数的耗时增长 ====')
sizes = [10000, 50000, 100000, 500000]
costs = []
for s in sizes:
    t0 = time.perf_counter()
    timed_heavy(s)
    costs.append((time.perf_counter() - t0) * 1000)
for s, c in zip(sizes, costs):
    print(f'数据量 {s:>7} -> 耗时 {c:7.1f} 毫秒')

# ==== 7. 画三张图 ====
print('\n==== 7. 画三张图:缓存对比 / 重试尝试 / 数据量耗时增长 ====')

# 图1:缓存前后耗时对比(第一次需联网/计算 vs 第二次命中缓存)
fig, ax = plt.subplots(figsize=(11, 6))
bars = ax.bar(['第一次(无缓存,需联网)', '第二次(命中缓存)'],
              [cost_first * 1000, cost_cached * 1000],
              color=['#d62728', '#2ca02c'])
ax.set_title(f'{NAME} 同一函数:缓存把耗时砍到接近零')
ax.set_ylabel('耗时(毫秒)')
for b, v in zip(bars, [cost_first * 1000, cost_cached * 1000]):
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/6)

# 装饰器与上下文管理器 一

## functools.wraps 写 @timer / @retry 重试 / lru\_cache 缓存 / contextmanager 写 with 计时块

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.text(b.get_x() + b.get_width() / 2, v, f'{v:.1f} ms', ha='center',
va='bottom')
ax.grid(alpha=0.3, axis='y'); plt.tight_layout(); plt.savefig('cache.png',
dpi=120); plt.close(fig)

# 图2:retry 重试尝试示意(前几次失败、第几次成功)
fig, ax = plt.subplots(figsize=(11, 6))
attempts = [a for a, _ in _attempt_log]
results = [1 if ok else 0 for _, ok in _attempt_log]
colors = ['#2ca02c' if r else '#d62728' for r in results]
ax.bar([f'第 {a} 次' for a in attempts], [1] * len(attempts), color=colors)
ax.set_title('@retry 重试:前 2 次失败(红)第 3 次成功(绿)')
ax.set_ylabel('一次尝试'); ax.set_yticks([])
ax.grid(alpha=0.3, axis='y'); plt.tight_layout(); plt.savefig('retry.png',
dpi=120); plt.close(fig)

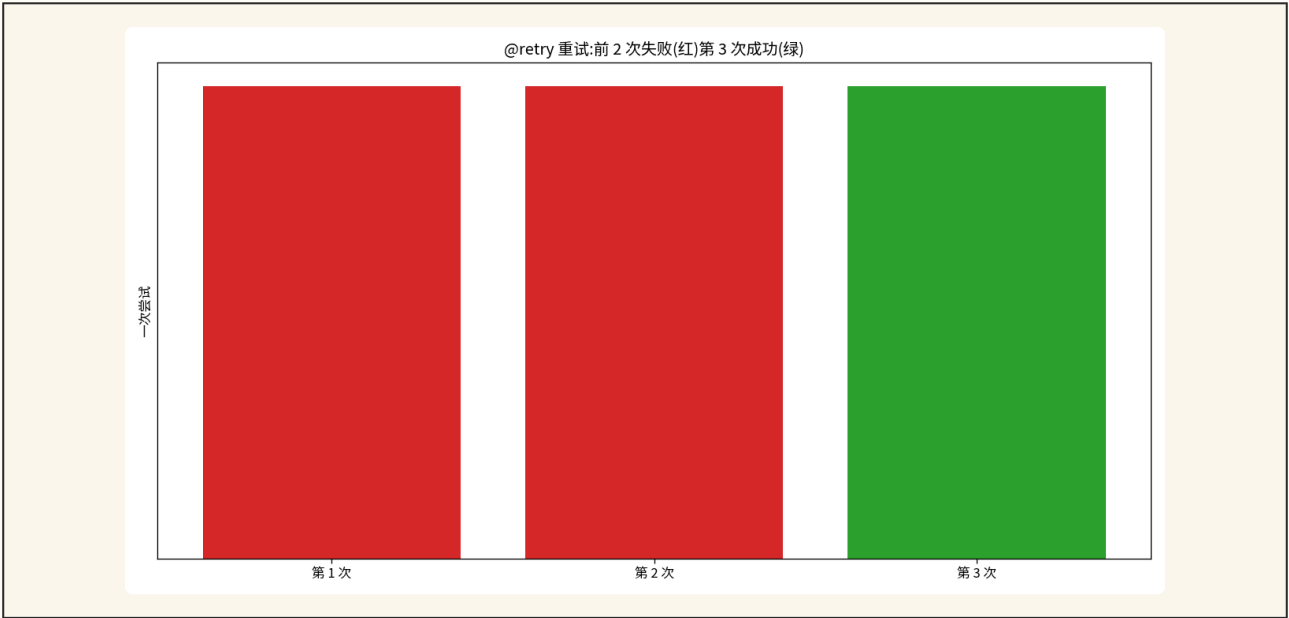
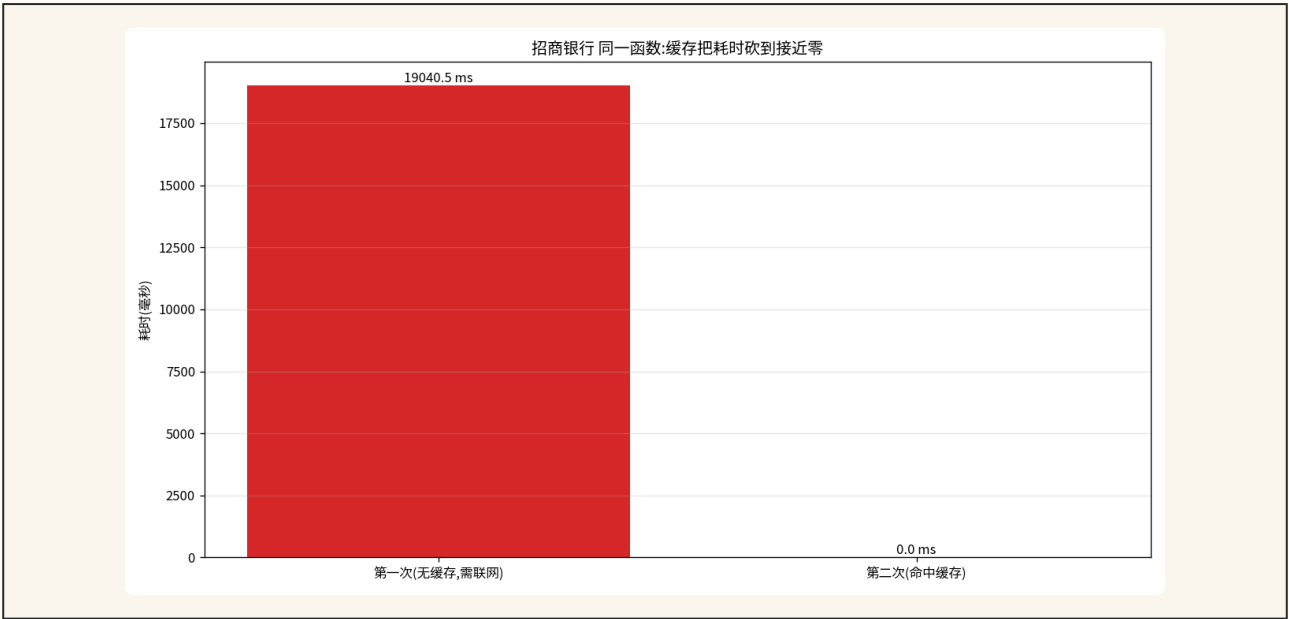
# 图3:数据量 vs 耗时增长曲线(@timer 测出)
fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(sizes, costs, 'o-', color='#1f77b4', lw=2, markersize=8)
for s, c in zip(sizes, costs):
    ax.text(s, c, f'{c:.0f} ms', ha='left', va='bottom')
ax.set_title('@timer 测:数据量越大,慢函数耗时越长')
ax.set_xlabel('数据量(数组长度)'); ax.set_ylabel('耗时(毫秒)')
ax.grid(alpha=0.3); plt.tight_layout(); plt.savefig('grow.png', dpi=120);
plt.close(fig)

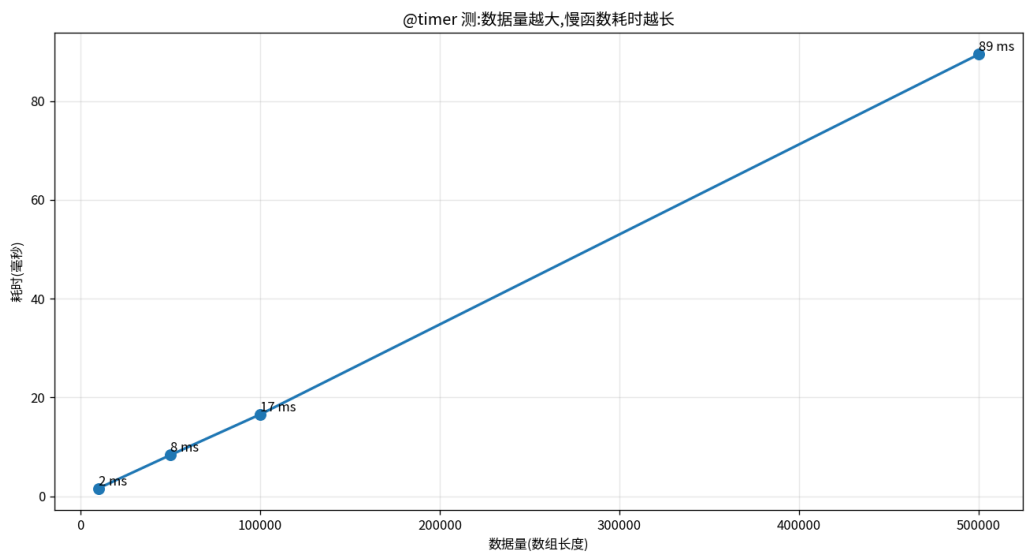
print('\n[done] 装饰器(timer/retry/lru_cache)+ with 上下文 + 3 张图全部完成')
```

05 · CHART + ANALYSIS · 看图说话

# 回测结果

// · matplotlib 真实输出





缓存效果(招商银行 sh.600036)

同一个拉招行日线+算均线的函数:第一次需联网约 19 秒,第二次命中 lru\_cache 仅 0.001 毫秒,几乎瞬间返回

@retry 重试演示

故意前 2 次失败、第 3 次成功;尝试记录 (1,失败)(2,失败)(3,成功),retry 自动扛过前两次网络抖动并最终拿到结果

@timer 测耗时随数据量增长

同一慢函数:数据量 1万 →1.6ms、5万→8.4ms、10万 →16.6ms、50万→89.5ms,耗时随数据量近似线性增长

with 计时块(进门开灯/出门关灯)

用 contextmanager 写的 timer\_block 包住一段 CPU 计算:进门自动开始计时、出门自动打印,实测约 17 毫秒,中间代码一字未改

本课用到的工具

functools.wraps 保留原名 / @timer 计时 / @retry 重试 / functools.lru\_cache 缓存 / contextlib.contextmanager 配 with 自动收尾

为什么 functools.wraps 重要

不加它,被装饰后函数的 \_\_name\_\_ 会变成 wrapper,原名丢失,看日志、debug 全对不上号;加上它套壳后仍保留原函数名

### ► 01 装饰器 = 给函数套个壳加功能,原码一字不改

装饰器最反直觉也最迷人的地方是,它能给一个函数加上全新的能力,却不动函数里面任何一行代码。就像给手机套个壳,壳上加了防摔功能,手机本身一个零件都没动。你只要在函数头上贴一个标签,它被调用时就自动多了计时、重试或缓存的能力。本课最值得记住的思路是:计时、重试、缓存这些跟业务无关、却到处都要的横切杂活,各做成一个外挂写一次,到处贴,正经逻辑就清清爽爽。老岳的脚本从抄了一地的重复代码瘦成一大圈,靠的就是这个。

### ► 02 `functools.wraps`:套壳后必须保住原名

写装饰器有个一定要记住的标配动作,在里层的 wrapper 上加一行 `@functools.wraps(func)`。原因是套壳之后外界看到的其实是壳,如果不加这一行,你去问函数叫什么名字,它会回答 wrapper,原来的名字就丢了。后果是看运行日志、排查问题时满屏都是 wrapper,你根本分不清哪个是哪个,debug 抓瞎。加上 `functools.wraps`,被装饰后的函数依然记得自己本来叫什么,名字和说明都保留。这一行几乎是写装饰器的第一件事,养成习惯就好,别等出了问题才想起来。

### ► 03 `@retry`:网络抖动自动再试,从崩溃边缘救回脚本

拉行情数据最烦的就是网络偶尔抖一下,一次没拉到整个脚本就崩了。`@retry` 装饰器给函数套上自动再试的能力:一旦失败就自动再来一次,试够指定次数还不行才真正报错。本课用一个故意前 2 次失败、第 3 次成功的函数演示,你能亲眼看到它老老实实记下第一次失败、第二次失败、第三次成功,主流程一个字都不用管。要点是 `retry` 只该针对网络超时这类临时故障,代码里真的 bug(变量名写错、参数传反)别用它无脑吞掉,逻辑错误要让它立刻报出来。

### ► 04 `lru_cache`:同样输入第二次直接秒回

有些计算很慢,比如联网拉一只票两年的日线再算均线。同样的参数算过一遍待会儿又要算,就是白等。`functools.lru_cache` 给函数加一个记忆,把每次的输入和结果记下来,下次遇到一模一样的输入直接端出上次的结果,根本不再算第二遍。本课用招商银行真实数据演示,第一次需联网约 19 秒,第二次命中缓存仅 0.001 毫秒,几乎瞬间返回,把耗时砍到接近零。注意它是把双刃剑:只缓存真正不变的东西,会随时间更新的行情别无脑缓存,否则明天数据变了还吃昨天的旧结果。

### ► 05 `with` 块:进门开灯、出门关灯,自动收尾绝不忘

上下文管理器就是那个 `with` 块,精髓是进门自动准备、出门自动收拾,就像走进房间自动开灯、离开自动关灯。用 `contextlib.contextmanager` 你能自己写一个:函数里一个 `yield` 把代码劈成两半,yield 之前是进门开灯(比如开始计时),yield 之后是出门关灯(比如打印耗时),`with` 块结束自动执行关灯那半。本课用它包住一段 CPU 计算,进门开始计时、出门自动打印约 17 毫秒,中间被包住的代码一字未改。关键是收尾逻辑要放进 `try finally` 的 `finally` 里,这样哪怕中间抛了异常,关灯那一下照样执行,文件、连接、计时绝不会漏掉收尾。

05 · REAL MARKETS · 真实市场案例

# A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

|      |           |                                                          |
|------|-----------|----------------------------------------------------------|
| 回测   |           | 给回测里每个慢函数贴一行 @timer,自动打印耗时,一眼看出哪步最拖时间                    |
| 数据拉取 | sh.600036 | 拉招商银行日线偶尔断网,套上 @retry(times=3) 自动重试,脚本不再因一次抖动整体崩溃        |
| 重复计算 |           | 同一组参数的均线/指标被反复调用,加 functools.lru_cache 缓存,第二次直接秒回省掉联网和计算 |
| 工程   |           | 用 with 块管理文件和数据库连接,进门自动打开、出门自动关闭,绝不要忘记释放资源               |



## 这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

### △ 01

#### 写装饰器忘了加 `functools.wraps`,函数名丢了

套壳之后如果不在里层加 `@functools.wraps(func)`,被装饰函数的 `__name__` 会变成 `wrapper`,原来的名字和说明全丢。以后看日志、debug、做文档都对不上号,排查问题特别费劲。养成习惯:写装饰器第一件事就是给里层函数加 `functools.wraps`。

### △ 02

#### `lru_cache` 缓存了会变的数据,行情更新了还吃旧缓存

缓存的前提是同样输入对应同样输出。如果你缓存的是今天的行情,明天数据更新了,`lru_cache` 还会把昨天的旧结果端给你,导致你拿着过期数据做决策。会随时间变化的数据要么别缓存、要么把日期当成参数的一部分,要么定期清缓存。缓存是把双刃剑,只缓存真正不变的东西。

### △ 03

#### `@retry` 把不该重试的错误也无脑重试

`retry` 适合重试网络抖动这种偶发故障,但如果是你代码里的 bug,比如变量名写错、参数传反,重试多少次都是同样的错,白白浪费时间还掩盖了真问题。重试应该只针对临时性的故障(网络、超时),逻辑错误要让它立刻报出来,别用 `retry` 一股脑全吞了。

### △ 04

#### `with` 块没正确释放资源,等于白用了上下文管理器

上下文管理器的价值就在出门自动收尾。自己写 `contextmanager` 时,如果收尾代码没放进 `finally`,一旦 `with` 块里抛了异常,关灯那半就被跳过,文件没关、连接没断、计时没停。写 `contextmanager` 一定把收尾逻辑放在 `try finally` 的 `finally` 里,保证无论出不出错都执行。

### △ 05

#### 装饰器叠太多,执行顺序搞混

一个函数上面可以叠好几个装饰器,比如又 `@timer` 又 `@retry`。它们的生效顺序是从下往上套壳、从上往下执行,叠多了很容易搞混谁先谁后,导致计时把重试的等待也算进去、或者缓存命中了却还在重试。叠装饰器时想清楚顺序,一般不要超过两三个,叠太多就该考虑换个写法了。

# 用装饰器和 with 写出干净代码的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 凡是跟业务无关、却到处都要的事(计时/重试/缓存/日志),抽成装饰器写一次到处贴
- 02 写装饰器第一件事就给里层函数加 `functools.wraps`,保住原函数的名字
- 03 `lru_cache` 只缓存真正不变的东西,会随时间更新的行情数据别无脑缓存
- 04 `retry` 只针对网络抖动这类临时故障,代码 bug 要让它立刻报出来
- 05 自己写 `contextmanager` 时收尾代码放进 `finally`,保证出门一定关灯

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

# 把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 装饰器 = 给函数套个壳加功能,在函数上面写一行 @timer,原函数一个字不用改。
- 02 functools.wraps 保住被装饰函数的原名字,不加它名字会变成 wrapper,debug 抓瞎。
- 03 @retry 让联网失败自动重试几次,前几次断了第几次成了脚本照常跑,不再一崩到底。
- 04 functools.lru\_cache 把结果缓存,同样输入第二次直接秒回,省掉联网和重复计算。
- 05 contextmanager 配 with 进门开灯、出门关灯,yield 一行把代码劈成准备和收尾两半。
- 06 计时/重试/缓存这些横切的事写一次到处复用,代码立刻干净10 倍,改一处全跟着变。

## 08 · SELF-CHECK · 自测题

# 答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

**Q01** 用『给手机套壳』打比方,解释什么是装饰器,以及为什么说它不改函数一行原码。

**Q02** 写装饰器为什么一定要加 `functools.wraps`? 不加会出什么问题?

**Q03** `lru_cache` 缓存很爽,但什么样的数据千万不能随便缓存?为什么?

**Q04** `with` 块的『进门开灯、出门关灯』指的是什么?`yield` 在自己写的上下文管理器里起什么作用?

## 09 · NEXT STEP · 下一步

# 继续往前走

// 看完这节,下面是延伸方向 + 明天预告

## 推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Ramalho 《Fluent Python》(2022/0'Reilly)– 装饰器、闭包与上下文管理器讲得最透彻的一本,例子地道。
- ▶ Beazley & Jones 《Python Cookbook》(2013/0'Reilly)– 第 9 章元编程,装饰器和 with 的实战配方一抓一大把。
- ▶ Slatkin 《Effective Python》(2019/Addison-Wesley)– 用 `functools.wraps`、用 `contextlib` 写 with 的条款讲清了为什么这么做。
- ▶ Python 官方 PEP 318 与 PEP 343([python.org](https://python.org))– 装饰器语法和 with 语句的官方设计文档,源头权威。
- ▶ Python 官方文档 `functools.lru_cache` 与 `contextlib`([docs.python.org](https://docs.python.org))– `lru_cache` 缓存和 `contextmanager` 的标准用法手册。

## NEXT LECTURE · 下一节预告

## DAY 052 asyncio 异步拉数据

学会了给函数套壳加功能,下一节我们解决一个更要命的痛点:拉数据太慢。同样要拉十只票的行情,一个个排队拉要等很久,用 `asyncio` 异步并发让它们一起开跑,十个任务几乎同时返回,速度能快10 倍。下节带你领略异步编程的威力。